MySQL Interview Questions with Queries

A focused basic-to-advanced interview handbook with concise definitions and practical query examples. Body font: 9 pt. A divider is included after every question.

1. MySQL and SQL Fundamentals

Q1. What is MySQL?

Definition

MySQL is an open-source relational database management system that stores structured data in tables and uses SQL to create, read, update, and delete records.

Query / Example

```
SELECT VERSION();
```

Q2. What is SQL?

Definition

SQL is the standard language used to define database structures, manipulate data, control access, and retrieve information from relational databases.

Query / Example

```
SELECT first_name, salary FROM employees;
```

Q3. What is the difference between SQL and MySQL?

Definition

SQL is a query language, while MySQL is a database management system that implements SQL and provides storage, security, transactions, and administration features.

Query / Example

```
SELECT DATABASE();
```

Q4. What is a database?

Definition

A database is an organized collection of related data that can be stored, searched, updated, and managed efficiently.

Query / Example

```
CREATE DATABASE company_db;
```

Q5. What is a table?

Definition

A table stores related data in rows and columns, where each row represents a record and each column represents an attribute.

Query / Example

```
CREATE TABLE employees (id INT, name VARCHAR(100));
```

Q6. What is a row and a column?

Definition

A row represents one complete record, while a column represents one specific field or attribute shared by all records in a table.

Query / Example

```
SELECT id, name FROM employees;
```

Q7. What is a schema in MySQL?

Definition

A schema is a logical container for database objects such as tables, views, indexes, procedures, and triggers; in MySQL, schema and database are effectively the same.

Query / Example

```
CREATE SCHEMA sales_db;
```

Q8. What are DDL commands?

Definition

DDL commands define or change database structures, including CREATE, ALTER, DROP, TRUNCATE, and RENAME.

Query / Example

```
ALTER TABLE employees ADD COLUMN email VARCHAR(150);
```

Q9. What are DML commands?

Definition

DML commands manipulate table data and mainly include INSERT, UPDATE, DELETE, and SELECT.

Query / Example

```
UPDATE employees SET salary = 60000 WHERE id = 10;
```

Q10. What are DCL commands?

Definition

DCL commands control database permissions and mainly include GRANT and REVOKE.

Query / Example

```
GRANT SELECT ON company_db.* TO 'report_user'@'localhost';
```

Q11. What are TCL commands?

Definition

TCL commands manage transactions and include COMMIT, ROLLBACK, and SAVEPOINT.

Query / Example

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
COMMIT;
```

Q12. What is the difference between DELETE, TRUNCATE, and DROP?

Definition

DELETE removes selected rows, TRUNCATE removes all rows quickly, and DROP removes the entire table structure together with its data.

Query / Example

```
DELETE FROM logs WHERE created_at < '2025-01-01';
TRUNCATE TABLE temp_logs;
DROP TABLE old_logs;
```

Q13. What is NULL?

Definition

NULL represents a missing, unknown, or unavailable value and must be tested using IS NULL or IS NOT NULL.

Query / Example

```
SELECT * FROM employees WHERE manager_id IS NULL;
```

Q14. What is the difference between NULL and an empty string?

Definition

NULL means no value is known, while an empty string is a known text value with zero characters.

Query / Example

```
SELECT * FROM users WHERE middle_name IS NULL OR middle_name = '';
```

Q15. What is the default MySQL port?

Definition

The default TCP port used by MySQL Server is 3306, although it can be changed in the server configuration.

Query / Example

```
SHOW VARIABLES LIKE 'port';
```

2. Data Types, Keys, and Constraints

Q16. What are common MySQL data types?

Definition

MySQL supports numeric, string, date/time, JSON, binary, spatial, and boolean-compatible data types for storing different forms of information.

Query / Example

```
CREATE TABLE products (
  id INT,
  price DECIMAL(10,2),
  name VARCHAR(100),
  created_at DATETIME
);
```

Q17. CHAR vs VARCHAR

Definition

CHAR uses fixed-length storage and is suitable for consistently sized values, while VARCHAR stores variable-length text more efficiently.

Query / Example

```
CREATE TABLE codes (country_code CHAR(2), description VARCHAR(255));
```

Q18. INT vs BIGINT

Definition

INT stores a smaller integer range using less space, while BIGINT supports much larger values and is suitable for high-volume identifiers.

Query / Example

```
CREATE TABLE events (id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY);
```

Q19. FLOAT vs DECIMAL

Definition

FLOAT stores approximate numbers, while DECIMAL stores exact fixed-point values and is preferred for financial calculations.

Query / Example

```
CREATE TABLE invoices (amount DECIMAL(12,2));
```

Q20. DATE vs DATETIME vs TIMESTAMP

Definition

DATE stores only a calendar date, DATETIME stores date and time, and TIMESTAMP stores a UTC-based value with a smaller supported range.

Query / Example

```
CREATE TABLE audit_log (
  event_date DATE,
  happened_at DATETIME,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Q21. What is a primary key?

Definition

A primary key uniquely identifies each row and automatically enforces uniqueness and non-null values.

Query / Example

```
CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(100));
```

Q22. What is a foreign key?

Definition

A foreign key links a child table to a parent table and helps enforce referential integrity between related records.

Query / Example

```
CREATE TABLE orders (
  id INT PRIMARY KEY,
  user_id INT,
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Q23. What is a unique key?

Definition

A unique key prevents duplicate values in a column or column combination, while still allowing NULL values depending on the design.

Query / Example

```
ALTER TABLE users ADD CONSTRAINT uq_users_email UNIQUE (email);
```

Q24. Primary key vs unique key

Definition

A table has only one primary key and it cannot contain NULL, while it can have multiple unique keys that may allow NULL values.

Query / Example

```
CREATE TABLE customers (
  id INT PRIMARY KEY,
  email VARCHAR(150) UNIQUE
);
```

Q25. What is a composite key?

Definition

A composite key uses two or more columns together to uniquely identify a row.

Query / Example

```
CREATE TABLE order_items (
  order_id INT,
  product_id INT,
  PRIMARY KEY (order_id, product_id)
);
```

Q26. What is AUTO_INCREMENT?

Definition

AUTO_INCREMENT automatically generates sequential numeric values for a key column when new rows are inserted.

Query / Example

```
CREATE TABLE employees (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100));
```

Q27. What is NOT NULL?

Definition

NOT NULL requires a column to always contain a value during insert or update operations.

Query / Example

```
ALTER TABLE employees MODIFY name VARCHAR(100) NOT NULL;
```

Q28. What is DEFAULT?

Definition

DEFAULT provides an automatic value when an insert statement does not specify a value for the column.

Query / Example

```
ALTER TABLE users ADD status VARCHAR(20) DEFAULT 'ACTIVE';
```

Q29. What is CHECK?

Definition

CHECK validates inserted or updated data against a condition and rejects rows that do not satisfy it.

Query / Example

```
CREATE TABLE employees (
  id INT PRIMARY KEY,
  age INT CHECK (age >= 18)
);
```

Q30. What is ON DELETE CASCADE?

Definition

ON DELETE CASCADE automatically deletes child rows when the referenced parent row is deleted.

Query / Example

```
FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE
```

Q31. What is ON DELETE SET NULL?

Definition

ON DELETE SET NULL sets the child foreign-key value to NULL when the referenced parent row is deleted.

Query / Example

```
FOREIGN KEY (manager_id) REFERENCES employees(id) ON DELETE SET NULL
```

---

3. SELECT, Filtering, Sorting, and Functions

Q32. How do you select all rows from a table?

Definition

SELECT retrieves rows and columns from one or more tables according to the specified filters and expressions.

Query / Example

```
SELECT * FROM employees;
```

Q33. How do you select specific columns?

Definition

List only the required columns to reduce unnecessary data transfer and improve readability.

Query / Example

```
SELECT id, name, salary FROM employees;
```

Q34. What is DISTINCT?

Definition

DISTINCT removes duplicate result rows based on the selected column combination.

Query / Example

```
SELECT DISTINCT department_id FROM employees;
```

Q35. What is WHERE?

Definition

WHERE filters rows before grouping and aggregation are performed.

Query / Example

```
SELECT * FROM employees WHERE salary > 50000;
```

Q36. What is ORDER BY?

Definition

ORDER BY sorts result rows in ascending or descending order using one or more columns.

Query / Example

```
SELECT * FROM employees ORDER BY salary DESC, name ASC;
```

Q37. What is LIMIT?

Definition

LIMIT restricts the number of rows returned and can be combined with OFFSET for pagination.

Query / Example

```
SELECT * FROM employees ORDER BY id LIMIT 10 OFFSET 20;
```

Q38. What is BETWEEN?

Definition

BETWEEN checks whether a value falls within an inclusive lower and upper range.

Query / Example

```
SELECT * FROM orders WHERE total BETWEEN 1000 AND 5000;
```

Q39. What is IN?

Definition

IN checks whether a value matches any value in a supplied list or subquery.

Query / Example

```
SELECT * FROM employees WHERE department_id IN (10, 20, 30);
```

Q40. What is LIKE?

Definition

LIKE performs pattern matching using percent for multiple characters and underscore for a single character.

Query / Example

```
SELECT * FROM customers WHERE name LIKE 'A%';
```

Q41. What is REGEXP?

Definition

REGEXP performs regular-expression matching for more advanced text searches.

Query / Example

```
SELECT * FROM users WHERE email REGEXP '^[A-Za-z0-9._%+-]+@';
```

Q42. What is CASE?

Definition

CASE adds conditional logic inside a query and returns different values based on evaluated conditions.

Query / Example

```
SELECT name,
CASE
  WHEN salary >= 80000 THEN 'High'
  WHEN salary >= 50000 THEN 'Medium'
  ELSE 'Low'
END AS salary_band
FROM employees;
```

Q43. What is COALESCE?

Definition

COALESCE returns the first non-NULL value from a list of expressions.

Query / Example

```
SELECT name, COALESCE(phone, mobile, 'Not available') AS contact FROM customers;
```

Q44. What is IFNULL?

Definition

IFNULL returns an alternative value when the first expression is NULL.

Query / Example

```
SELECT name, IFNULL(bonus, 0) AS bonus FROM employees;
```

Q45. What is CONCAT?

Definition

CONCAT joins multiple strings into one result.

Query / Example

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM employees;
```

Q46. What is SUBSTRING?

Definition

SUBSTRING extracts part of a string starting at a given position for an optional length.

Query / Example

```
SELECT SUBSTRING(phone, 1, 5) AS prefix FROM customers;
```

Q47. What is DATE_FORMAT?

Definition

DATE_FORMAT converts a date or datetime into a formatted text representation.

Query / Example

```
SELECT DATE_FORMAT(created_at, '%d-%m-%Y') AS created_date FROM orders;
```

Q48. How do you get the current date and time?

Definition

MySQL provides CURRENT_DATE, CURRENT_TIME, and NOW to return current temporal values.

Query / Example

```
SELECT CURRENT_DATE(), CURRENT_TIME(), NOW();
```

Q49. What is DATEDIFF?

Definition

DATEDIFF returns the number of days between two date values.

Query / Example

```
SELECT DATEDIFF(CURDATE(), joining_date) AS days_employed FROM employees;
```

4. Aggregate Functions and Grouping

Q50. What are aggregate functions?

Definition

Aggregate functions calculate a single result from multiple rows, such as COUNT, SUM, AVG, MIN, and MAX.

Query / Example

```
SELECT COUNT(*), SUM(salary), AVG(salary), MIN(salary), MAX(salary) FROM employees;
```

Q51. COUNT(*) vs COUNT(column)

Definition

COUNT(*) counts every row, while COUNT(column) ignores rows where the specified column is NULL.

Query / Example

```
SELECT COUNT(*), COUNT(manager_id) FROM employees;
```

Q52. What is GROUP BY?

Definition

GROUP BY combines rows with the same values so aggregate functions can be calculated per group.

Query / Example

```
SELECT department_id, AVG(salary) FROM employees GROUP BY department_id;
```

Q53. What is HAVING?

Definition

HAVING filters grouped or aggregated results after GROUP BY has been evaluated.

Query / Example

```
SELECT department_id, COUNT(*) AS employee_count
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

Q54. WHERE vs HAVING

Definition

WHERE filters individual rows before grouping, while HAVING filters grouped results after aggregation.

Query / Example

```
SELECT department_id, AVG(salary)
FROM employees
WHERE status = 'ACTIVE'
GROUP BY department_id
HAVING AVG(salary) > 50000;
```

Q55. How do you find duplicate values?

Definition

Group by the target column and use HAVING COUNT greater than one to identify duplicates.

Query / Example

```
SELECT email, COUNT(*)
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Q56. How do you find the second-highest salary?

Definition

Sort distinct salaries in descending order and use LIMIT with an offset to return the second value.

Query / Example

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

Q57. How do you find the nth-highest salary?

Definition

Use DISTINCT with descending order and set the offset to n minus one.

Query / Example

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 4;
```

Q58. How do you find the highest salary per department?

Definition

Group rows by department and apply MAX to the salary column.

Query / Example

```
SELECT department_id, MAX(salary) AS highest_salary
FROM employees
GROUP BY department_id;
```

Q59. How do you find departments with no employees?

Definition

Use a left join from departments to employees and filter groups whose employee count is zero.

Query / Example

```
SELECT d.id, d.name
FROM departments d
LEFT JOIN employees e ON e.department_id = d.id
GROUP BY d.id, d.name
HAVING COUNT(e.id) = 0;
```

---

5. Joins and Set Operations

Q60. What is an INNER JOIN?

Definition

INNER JOIN returns only rows that have matching values in both joined tables.

Query / Example

```
SELECT e.name, d.name AS department
FROM employees e
INNER JOIN departments d ON d.id = e.department_id;
```

Q61. What is a LEFT JOIN?

Definition

LEFT JOIN returns every row from the left table and matching rows from the right table, using NULL when no match exists.

Query / Example

```
SELECT c.name, o.id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id;
```

Q62. What is a RIGHT JOIN?

Definition

RIGHT JOIN returns every row from the right table and matching rows from the left table.

Query / Example

```
SELECT e.name, d.name
FROM employees e
RIGHT JOIN departments d ON d.id = e.department_id;
```

Q63. Does MySQL support FULL OUTER JOIN?

Definition

MySQL does not provide FULL OUTER JOIN directly, but it can be simulated by combining left and right joins with UNION.

Query / Example

```
SELECT a.id, b.id
FROM a LEFT JOIN b ON a.id = b.id
UNION
SELECT a.id, b.id
FROM a RIGHT JOIN b ON a.id = b.id;
```

Q64. What is a CROSS JOIN?

Definition

CROSS JOIN returns the Cartesian product of two tables, combining every row from the first table with every row from the second.

Query / Example

```
SELECT c.color, s.size
FROM colors c CROSS JOIN sizes s;
```

Q65. What is a SELF JOIN?

Definition

A self join joins a table to itself and is commonly used for hierarchical relationships such as employees and managers.

Query / Example

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON m.id = e.manager_id;
```

Q66. JOIN vs subquery

Definition

A join combines tables directly and is often easier for relational retrieval, while a subquery produces an intermediate result used by another query.

Query / Example

```
SELECT e.name
FROM employees e
JOIN departments d ON d.id = e.department_id
WHERE d.name = 'Sales';
```

Q67. What is UNION?

Definition

UNION combines result sets and removes duplicates, provided the queries return the same number of compatible columns.

Query / Example

```
SELECT email FROM customers
UNION
SELECT email FROM suppliers;
```

Q68. UNION vs UNION ALL

Definition

UNION removes duplicates, while UNION ALL keeps duplicates and is generally faster.

Query / Example

```
SELECT city FROM customers
UNION ALL
SELECT city FROM suppliers;
```

Q69. How do you find unmatched rows?

Definition

Use a left join and filter rows where the matching right-side key is NULL.

Query / Example

```
SELECT c.*
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL;
```

---

6. Subqueries, CTEs, and Window Functions

Q70. What is a subquery?

Definition

A subquery is a query nested inside another query and can return a scalar value, row, column, or table-like result.

Query / Example

```
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Q71. What is a correlated subquery?

Definition

A correlated subquery references columns from the outer query and is evaluated for each outer row.

Query / Example

```
SELECT e1.*
FROM employees e1
WHERE salary > (
  SELECT AVG(e2.salary)
  FROM employees e2
  WHERE e2.department_id = e1.department_id
);
```

Q72. EXISTS vs IN

Definition

EXISTS checks whether a subquery returns any row and is often efficient for correlated checks, while IN compares a value against a result list.

Query / Example

```
SELECT * FROM customers c
WHERE EXISTS (
  SELECT 1 FROM orders o WHERE o.customer_id = c.id
);
```

Q73. What is a CTE?

Definition

A Common Table Expression is a named temporary result available to a single statement and improves readability of complex queries.

Query / Example

```
WITH high_paid AS (
  SELECT * FROM employees WHERE salary > 80000
)
SELECT * FROM high_paid;
```

Q74. What is a recursive CTE?

Definition

A recursive CTE repeatedly references itself and is useful for hierarchical data, sequences, and tree traversal.

Query / Example

```
WITH RECURSIVE nums AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM nums WHERE n < 10
)
SELECT * FROM nums;
```

Q75. What are window functions?

Definition

Window functions calculate values across related rows without collapsing them into one row per group.

Query / Example

```
SELECT name, salary,
AVG(salary) OVER (PARTITION BY department_id) AS dept_avg
FROM employees;
```

Q76. What is ROW_NUMBER?

Definition

ROW_NUMBER assigns a unique sequential number to rows within an ordered partition.

Query / Example

```
SELECT name, department_id,
ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rn
FROM employees;
```

Q77. RANK vs DENSE_RANK

Definition

RANK leaves gaps after ties, while DENSE_RANK gives tied rows the same rank without gaps.

Query / Example

```
SELECT name, salary,
RANK() OVER (ORDER BY salary DESC) AS rnk,
DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk
FROM employees;
```

Q78. How do you get the top three salaries per department?

Definition

Use a ranking window function partitioned by department and filter ranks up to three.

Query / Example

```
WITH ranked AS (
  SELECT e.*,
  DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rnk
  FROM employees e
)
SELECT * FROM ranked WHERE rnk <= 3;
```

Q79. What are LAG and LEAD?

Definition

LAG reads a previous row value and LEAD reads a following row value within a defined window.

Query / Example

```
SELECT order_date, total,
LAG(total) OVER (ORDER BY order_date) AS previous_total
FROM orders;
```

---

7. Indexes and Performance

Q80. What is an index?

Definition

An index is a separate data structure that speeds up row lookup and sorting at the cost of extra storage and write overhead.

Query / Example

```
CREATE INDEX idx_employees_email ON employees(email);
```

Q81. What is a clustered index in InnoDB?

Definition

InnoDB stores table rows in primary-key order, so the primary key acts as the clustered index and secondary indexes reference it.

Query / Example

```
ALTER TABLE employees ADD PRIMARY KEY (id);
```

Q82. What is a composite index?

Definition

A composite index includes multiple columns and is most effective when queries use its leftmost column sequence.

Query / Example

```
CREATE INDEX idx_orders_customer_date ON orders(customer_id, order_date);
```

Q83. What is the leftmost-prefix rule?

Definition

A composite index can efficiently support queries that use the leading indexed columns in order, but not usually queries that skip the first column.

Query / Example

```
CREATE INDEX idx_name_city ON customers(last_name, city);
SELECT * FROM customers WHERE last_name = 'Kumar';
```

Q84. What is a covering index?

Definition

A covering index contains all columns required by a query, allowing MySQL to return data without reading the table rows.

Query / Example

```
CREATE INDEX idx_users_email_status ON users(email, status);
```

Q85. How do you view a query execution plan?

Definition

EXPLAIN shows how MySQL plans to access tables, indexes, join order, and estimated rows.

Query / Example

```
EXPLAIN SELECT * FROM employees WHERE email = 'a@b.com';
```

Q86. What is EXPLAIN ANALYZE?

Definition

EXPLAIN ANALYZE executes the query and reports actual timing, rows, loops, and execution details.

Query / Example

```
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 10;
```

Q87. Why may MySQL ignore an index?

Definition

MySQL may ignore an index when the condition is not selective, a function is applied to the column, types do not match, or a full scan is cheaper.

Query / Example

```
SELECT * FROM users WHERE LOWER(email) = 'a@b.com';
```

Q88. How can functions prevent index usage?

Definition

Applying a function to an indexed column can make the condition non-sargable because the stored index values no longer match the expression directly.

Query / Example

```
SELECT * FROM orders
WHERE order_date >= '2026-01-01'
  AND order_date < '2027-01-01';
```

Q89. How do you optimize a slow query?

Definition

Review the execution plan, reduce selected columns, add suitable indexes, avoid unnecessary scans, optimize joins, and limit result sizes.

Query / Example

```
EXPLAIN ANALYZE
SELECT id, name FROM employees
WHERE department_id = 10
ORDER BY salary DESC
LIMIT 20;
```

Q90. What is table partitioning?

Definition

Partitioning divides a large table into logical pieces based on a key, allowing some queries and maintenance tasks to access less data.

Query / Example

```
CREATE TABLE sales (
  id BIGINT,
  sale_date DATE
)
PARTITION BY RANGE (YEAR(sale_date)) (
  PARTITION p2025 VALUES LESS THAN (2026),
  PARTITION pmax VALUES LESS THAN MAXVALUE
);
```

Q91. What is ANALYZE TABLE?

Definition

ANALYZE TABLE updates table and index statistics used by the query optimizer.

Query / Example

```
ANALYZE TABLE orders;
```

---

8. Transactions, Locking, and Isolation

Q92. What is a transaction?

Definition

A transaction is a group of database operations treated as one logical unit that either completes fully or is rolled back.

Query / Example

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
```

Q93. What are ACID properties?

Definition

ACID means Atomicity, Consistency, Isolation, and Durability, which together ensure reliable transaction processing.

Query / Example

```
START TRANSACTION;
-- multiple related statements
COMMIT;
```

Q94. What is COMMIT?

Definition

COMMIT permanently saves all changes made in the current transaction.

Query / Example

```
COMMIT;
```

Q95. What is ROLLBACK?

Definition

ROLLBACK cancels uncommitted changes made in the current transaction.

Query / Example

```
ROLLBACK;
```

Q96. What is a SAVEPOINT?

Definition

A SAVEPOINT creates a named position inside a transaction so changes can be partially rolled back.

Query / Example

```
START TRANSACTION;
SAVEPOINT before_update;
UPDATE products SET price = price * 1.10;
ROLLBACK TO before_update;
```

Q97. What are transaction isolation levels?

Definition

Isolation levels control how transactions see each other's uncommitted or committed changes and balance consistency against concurrency.

Query / Example

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Q98. What is a dirty read?

Definition

A dirty read occurs when one transaction reads changes made by another transaction before those changes are committed.

Query / Example

```
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

Q99. What is a non-repeatable read?

Definition

A non-repeatable read occurs when the same row returns different committed values within one transaction.

Query / Example

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Q100. What is a phantom read?

Definition

A phantom read occurs when repeating a range query returns newly inserted or removed rows from another transaction.

Query / Example

```
SELECT * FROM orders WHERE total > 1000 FOR UPDATE;
```

Q101. What is a deadlock?

Definition

A deadlock occurs when transactions wait on each other's locked resources, forcing MySQL to roll back one transaction.

Query / Example

```
SHOW ENGINE INNODB STATUS;
```

Q102. What is SELECT FOR UPDATE?

Definition

SELECT FOR UPDATE locks selected rows until the transaction ends, preventing conflicting updates.

Query / Example

```
START TRANSACTION;
SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;
```

Q103. Optimistic vs pessimistic locking

Definition

Optimistic locking detects conflicts using a version value during update, while pessimistic locking blocks conflicts by locking rows first.

Query / Example

```
UPDATE products
SET stock = stock - 1, version = version + 1
WHERE id = 10 AND version = 3;
```

---

9. Normalization and Database Design

Q104. What is normalization?

Definition

Normalization organizes data to reduce duplication, prevent update anomalies, and improve data integrity.

Query / Example

```
CREATE TABLE departments (id INT PRIMARY KEY, name VARCHAR(100));
```

Q105. What is First Normal Form?

Definition

First Normal Form requires atomic column values and no repeating groups.

Query / Example

```
CREATE TABLE order_items (order_id INT, product_id INT, quantity INT);
```

Q106. What is Second Normal Form?

Definition

Second Normal Form requires First Normal Form and removes partial dependency on part of a composite key.

Query / Example

```
PRIMARY KEY (order_id, product_id)
```

Q107. What is Third Normal Form?

Definition

Third Normal Form requires Second Normal Form and removes transitive dependencies between non-key columns.

Query / Example

```
employees(department_id) REFERENCES departments(id)
```

Q108. What is BCNF?

Definition

BCNF is a stronger form of Third Normal Form in which every determinant must be a candidate key.

Query / Example

```
-- Redesign tables so every determinant is a candidate key.
```

Q109. What is denormalization?

Definition

Denormalization intentionally duplicates or combines data to improve read performance at the cost of additional maintenance.

Query / Example

```
ALTER TABLE orders ADD customer_name VARCHAR(100);
```

Q110. What is a surrogate key?

Definition

A surrogate key is an artificial identifier such as an auto-increment integer that has no business meaning.

Query / Example

```
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY
```

Q111. What is a natural key?

Definition

A natural key is a real business value that uniquely identifies a record, such as a tax number or email address.

Query / Example

```
UNIQUE KEY uq_customers_tax_id (tax_id)
```

Q112. One-to-one relationship

Definition

A one-to-one relationship links one row in a table to at most one row in another table.

Query / Example

```
CREATE TABLE profiles (
  user_id INT PRIMARY KEY,
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Q113. One-to-many relationship

Definition

A one-to-many relationship allows one parent row to be linked to many child rows.

Query / Example

```
orders.customer_id REFERENCES customers.id
```

Q114. Many-to-many relationship

Definition

A many-to-many relationship is implemented using a junction table containing foreign keys to both related tables.

Query / Example

```
CREATE TABLE student_courses (
  student_id INT,
  course_id INT,
  PRIMARY KEY (student_id, course_id)
);
```

---

10. Views, Procedures, Functions, and Triggers

Q115. What is a view?

Definition

A view is a saved SELECT statement that presents data like a virtual table without normally storing the result rows.

Query / Example

```
CREATE VIEW active_users AS
SELECT id, name, email FROM users WHERE status = 'ACTIVE';
```

Q116. What is a materialized view in MySQL?

Definition

MySQL does not provide native materialized views, but similar behavior can be implemented using summary tables refreshed by jobs.

Query / Example

```
CREATE TABLE daily_sales_summary AS
SELECT sale_date, SUM(total) total
FROM sales GROUP BY sale_date;
```

Q117. What is a stored procedure?

Definition

A stored procedure is a named group of SQL statements stored on the server and executed using CALL.

Query / Example

```
DELIMITER //
CREATE PROCEDURE GetUsers()
BEGIN
  SELECT * FROM users;
END //
DELIMITER ;
CALL GetUsers();
```

Q118. What is a stored function?

Definition

A stored function returns one value and can be used inside SQL expressions.

Query / Example

```
CREATE FUNCTION FullName(f VARCHAR(50), l VARCHAR(50))
RETURNS VARCHAR(101) DETERMINISTIC
RETURN CONCAT(f, ' ', l);
```

Q119. Procedure vs function

Definition

A procedure is invoked with CALL and may return result sets or OUT parameters, while a function returns one value and can be used in expressions.

Query / Example

```
SELECT FullName(first_name, last_name) FROM employees;
```

Q120. What is a trigger?

Definition

A trigger automatically runs before or after an INSERT, UPDATE, or DELETE event on a table.

Query / Example

```
CREATE TRIGGER users_before_insert
BEFORE INSERT ON users
FOR EACH ROW SET NEW.created_at = NOW();
```

Q121. What are OLD and NEW in triggers?

Definition

OLD refers to the previous row values and NEW refers to the inserted or updated row values inside a trigger.

Query / Example

```
INSERT INTO employee_audit(employee_id, old_salary, new_salary)
VALUES (OLD.id, OLD.salary, NEW.salary);
```

Q122. What is an event scheduler?

Definition

The event scheduler runs SQL statements automatically at specified times or intervals.

Query / Example

```
CREATE EVENT delete_old_logs
ON SCHEDULE EVERY 1 DAY
DO DELETE FROM logs WHERE created_at < NOW() - INTERVAL 90 DAY;
```

---

11. Security, Backup, and Administration

Q123. How do you create a MySQL user?

Definition

CREATE USER adds an account with a host restriction and authentication credentials.

Query / Example

```
CREATE USER 'app_user'@'localhost' IDENTIFIED BY 'StrongPassword!';
```

Q124. How do you grant permissions?

Definition

GRANT assigns specified privileges on database objects to a user account.

Query / Example

```
GRANT SELECT, INSERT, UPDATE ON company_db.* TO 'app_user'@'localhost';
```

Q125. How do you revoke permissions?

Definition

REVOKE removes previously granted privileges from a user account.

Query / Example

```
REVOKE DELETE ON company_db.* FROM 'app_user'@'localhost';
```

Q126. What is the principle of least privilege?

Definition

Least privilege means granting only the minimum permissions required for a user or application to perform its work.

Query / Example

```
GRANT SELECT ON reporting_db.* TO 'report_user'@'localhost';
```

Q127. How do you prevent SQL injection?

Definition

Use parameterized queries or prepared statements, validate inputs, and avoid concatenating untrusted data into SQL strings.

Query / Example

```
PREPARE stmt FROM 'SELECT * FROM users WHERE email = ?';
```

Q128. What is a prepared statement?

Definition

A prepared statement compiles SQL separately from data values, improving safety and sometimes reuse.

Query / Example

```
PREPARE stmt FROM 'SELECT * FROM users WHERE id = ?';
SET @id = 10;
EXECUTE stmt USING @id;
```

Q129. How do you back up a MySQL database?

Definition

mysqldump exports database structure and data into a SQL file that can later be restored.

Query / Example

```
mysqldump -u root -p company_db > company_db.sql
```

Q130. How do you restore a MySQL backup?

Definition

A SQL dump can be restored by sending it to the mysql client for the target database.

Query / Example

```
mysql -u root -p company_db < company_db.sql
```

Q131. What is replication?

Definition

Replication copies data changes from a source server to one or more replica servers for availability, scaling, or reporting.

Query / Example

```
SHOW REPLICA STATUS;
```

Q132. What is connection pooling?

Definition

Connection pooling reuses a limited set of database connections instead of opening a new connection for every request.

Query / Example

```
-- Configure pool size in the application database driver.
```

12. Practical Query Scenarios

Q133. Find employees earning above the company average

Definition

Compare each employee's salary with a scalar subquery that calculates the overall average salary.

Query / Example

```
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Q134. Find employees earning above their department average

Definition

Use a correlated subquery or window function to compare each salary with its department average.

Query / Example

```
SELECT * FROM (
  SELECT e.*,
  AVG(salary) OVER (PARTITION BY department_id) AS dept_avg
  FROM employees e
) x
WHERE salary > dept_avg;
```

Q135. Find customers who never placed an order

Definition

Use a left join and filter rows with no matching order.

Query / Example

```
SELECT c.*
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL;
```

Q136. Find duplicate emails and keep only the smallest ID

Definition

Use ROW_NUMBER to rank duplicates, then delete rows whose rank is greater than one.

Query / Example

```
WITH ranked AS (
  SELECT id,
  ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn
  FROM users
)
DELETE FROM users
WHERE id IN (SELECT id FROM ranked WHERE rn > 1);
```

Q137. Calculate a running total

Definition

Use SUM as a window function ordered by date or another sequence column.

Query / Example

```
SELECT order_date, total,
SUM(total) OVER (ORDER BY order_date, id) AS running_total
FROM orders;
```

Q138. Calculate month-wise sales

Definition

Group rows by formatted year and month and sum the sales amount.

Query / Example

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
SUM(total) AS sales
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

Q139. Find the latest order for each customer

Definition

Rank orders by date within each customer and keep the first row.

Query / Example

```
WITH ranked AS (
  SELECT o.*,
  ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC, id DESC) AS rn
  FROM orders o
)
SELECT * FROM ranked WHERE rn = 1;
```

Q140. Find consecutive login days

Definition

Use window functions to create groups based on the difference between login date and row number.

Query / Example

```
WITH x AS (
  SELECT user_id, login_date,
  DATE_SUB(login_date, INTERVAL ROW_NUMBER() OVER (
    PARTITION BY user_id ORDER BY login_date
  ) DAY) AS grp
  FROM logins
)
SELECT user_id, MIN(login_date), MAX(login_date), COUNT(*)
FROM x
GROUP BY user_id, grp;
```

Q141. Swap two column values

Definition

Use a single UPDATE statement so both values are assigned from the original row values.

Query / Example

```
UPDATE employees
SET first_name = last_name,
    last_name = first_name
WHERE id = 10;
```

Q142. Delete old records in batches

Definition

Delete a limited number of old rows repeatedly to reduce long locks and transaction size.

Query / Example

```
DELETE FROM logs
WHERE created_at < NOW() - INTERVAL 1 YEAR
LIMIT 10000;
```

Q143. Find missing numeric IDs

Definition

Generate or join a sequence and compare it against the table to identify absent values.

Query / Example

```
WITH RECURSIVE seq AS (
  SELECT 1 n
  UNION ALL
  SELECT n + 1 FROM seq WHERE n < 100
)
SELECT n FROM seq
LEFT JOIN employees e ON e.id = n
WHERE e.id IS NULL;
```

Q144. Update one table from another table

Definition

Use an UPDATE JOIN to copy or calculate values based on matching rows.

Query / Example

```
UPDATE employees e
JOIN salary_updates s ON s.employee_id = e.id
SET e.salary = s.new_salary;
```

Q145. Insert rows while avoiding duplicates

Definition

Use INSERT IGNORE or ON DUPLICATE KEY UPDATE when a unique key may already exist.

Query / Example

```
INSERT INTO users(email, name)
VALUES ('a@b.com', 'A')
ON DUPLICATE KEY UPDATE name = VALUES(name);
```

Q146. Return records created today

Definition

Filter a datetime column using a half-open date range to preserve index usage.

Query / Example

```
SELECT * FROM orders
WHERE created_at >= CURDATE()
  AND created_at < CURDATE() + INTERVAL 1 DAY;
```

Q147. Return records from the last seven days

Definition

Compare the date column with the current timestamp minus a seven-day interval.

Query / Example

```
SELECT * FROM orders
WHERE created_at >= NOW() - INTERVAL 7 DAY;
```
